

Graph Exploration Random Walks

Contents

- 16.1. Introduction
- 16.2. How Do Random Walks Work?
- 16.3. Implementation with NumPy
- 16.4. Visual Example
- 16.5. Testing Our Random Walk Algorithm
- 16.6. Visualizing the Random Walk Process

[Saltar al contenido principal](#)

- 16.7. Practical Applications of Random Walks
- 16.8. Conclusion
- 16.9. Exercise
- 16.10. Submission

16.1. Introducción

Al trabajar con grafos, los Paseos Aleatorios ofrecen una estrategia alternativa de exploración que imita el movimiento aleatorio a través de una red. En este cuaderno, implementaremos Caminatas Aleatorias usando NumPy y NetworkX para explorar un grafo no ponderado y no dirigido y, potencialmente, encontrar un camino entre nodos.

16.2. ¿Cómo funcionan los

[Saltar al contenido principal](#)

paseos aleatorios?

Los Paseos Aleatorios exploran un grafo moviéndose aleatoriamente de un nodo a otro:

1. Comenzamos en el nodo fuente y lo marcamos como visitado
2. En cada paso, seleccionamos aleatoriamente a uno de los vecinos del nodo actual
3. Nos mudamos a ese vecino y lo añadimos a nuestro camino
4. Este proceso continúa hasta que alcanzamos el nodo objetivo o superemos un número máximo de pasos

A diferencia de BFS y DFS, los Paseos Aleatorios no garantizan encontrar el camino más corto ni siquiera ningún

[Saltar al contenido principal](#)

propiedades de grafos, simular procesos estocásticos y pueden ser efectivos en grafos muy grandes donde la exploración sistemática es poco práctica.

16.3. Implementación con NumPy

Recuerda que está prohibido usar funciones o librerías integradas que implementen Paseos Aleatorios. Necesitas implementar el algoritmo desde cero usando NumPy y listas estándar de Python. Cualquier implementación que utilice funciones de Paseo Aleatorio integradas será considerada inválida.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

def random_walk_numpy(graph, source
```

[Saltar al contenido principal](#)

Implementation of Random Walk u

Parameters:

- graph: NetworkX graph (undire
- source: Source node
- target: Target node
- max_steps: Maximum number of
- seed: Random seed for reprodu

Returns:

- steps: Number of steps needed
 - path: List of nodes in the pa
 - success: Boolean indicating w
- """

`return steps, node_path, succes`

16.4. Ejemplo visual

Vamos a crear un grafo no dirigido sencillo para visualizar cómo funciona el algoritmo:



```
def create_example_graph():
```

[Saltar al contenido principal](#)

```

# Add nodes (cities)
cities = ['Madrid', 'Barcelona']
G.add_nodes_from(cities)

# Add edges (no weights)
edges = [
    ('Madrid', 'Barcelona'),
    ('Madrid', 'Valencia'),
    ('Madrid', 'Sevilla'),
    ('Madrid', 'Bilbao'),
    ('Madrid', 'Zaragoza'),
    ('Barcelona', 'Valencia'),
    ('Barcelona', 'Zaragoza'),
    ('Valencia', 'Sevilla'),
    ('Zaragoza', 'Bilbao')
]

# Add the edges
G.add_edges_from(edges)

return G

# Create and visualize the graph
G = create_example_graph()

# Visualize the graph
pos = nx.spring_layout(G, seed=123)

```

[Saltar al contenido principal](#)

```
nx.draw(G, pos, with_labels=True, n
plt.title("Undirected Graph of Span
plt.show()
```

16.5. Probando nuestro algoritmo de caminata aleatoria

Vamos a ejecutar nuestro algoritmo en este gráfico y ver los resultados:

```
# Define source and target nodes
source = 'Bilbao'
target = 'Sevilla'

# Run Random Walk
rw_steps, rw_path, success = random

if success:
    print(f"Random Walk from {source} to {target}")
    print(f"Number of steps: {rw_steps}")
    print(f"Path found: {rw_path}")
    print(f"Path length: {len(rw_path)}")
```

[Saltar al contenido principal](#)

```

        print(f"Partial path: {rw_path}")

# Compare with NetworkX (shortest path)
shortest_path = nx.shortest_path(G,
print(f"\nNetworkX result (shortest
print(f"Shortest path: {shortest_pa
print(f"Path length: {len(shortest_

# Check if Random Walk found the sh
if success and len(rw_path) == len(
    print("\nRandom Walk found the
elif success:
    print(f"\nRandom Walk found a v

```

16.6. Visualizing the Random Walk Process

Let's visualize the path that the Random Walk finds:

```

def visualize_random_walk_path(G, p
    """Visualizes the random walk p
    plt.figure(figsize=(10, 8))

```

[Saltar al contenido principal](#)


```

H = G.copy()
pos = nx.spring_layout(H, seed=

# Create directed edges for the
path_edges = [(path[i], path[i+

# Draw all nodes
nx.draw_networkx_nodes(H, pos,

# Highlight the nodes in the pa
nx.draw_networkx_nodes(H, pos,

# Draw all edges in gray
nx.draw_networkx_edges(H, pos,

# Draw the random walk path wit
nx.draw_networkx_edges(
    H, pos,
    edgelist=path_edges,
    edge_color='red',
    width=2.0,
    arrows=True, # Show arrows
    arrowstyle='->', # Arrow
    arrowsize=15,
    connectionstyle='arc3,rad=0
)

# Add numbers to show the order

```

[Saltar al contenido principal](#)

```
nx.draw_networkx_edge_labels(
    H, pos,
    edge_labels=edge_labels,
    font_size=10,
    font_weight='bold',
    bbox=dict(facecolor='white'
)

# Draw node labels
nx.draw_networkx_labels(H, pos)

plt.title(title)
plt.axis('off')
plt.show()

# Visualize the Random Walk path with
if success:
    visualize_random_walk_path(G, r
```

16.7. Practical Applications of Random Walks

Random Walks have many practical applications in various fields:

[Saltar al contenido principal](#)

1. **PageRank Algorithm:** Google's original algorithm uses random walks to rank web pages based on the probability of a random surfer visiting them.
2. **Recommendation Systems:** Random walks on user-item graphs can be used to generate personalized recommendations.
3. **Network Sampling:** Random walks provide a way to sample nodes from large networks when complete exploration is impractical.
4. **Community Detection:** Random walk-based metrics can identify densely connected communities in networks.

16.8. Conclusion

using NumPy and NetworkX. Random Walks provide a stochastic approach to graph traversal that differs fundamentally from deterministic methods like BFS and DFS. While random walks do not guarantee finding the shortest path or even any path at all, they have several advantages:

1. They require minimal memory as they only need to track the current position and path
2. They can be effective for exploring very large graphs where systematic traversal is impractical
3. They model many natural and social processes that involve random movement
4. They form the basis for many important algorithms in

[Saltar al contenido principal](#)

recommendation systems, web ranking, and network analysis

Our implementation demonstrates how random walks behave in graphs and highlights the trade-off between deterministic exploration and stochastic sampling. The visualization of the walk paths provides an intuitive understanding of how randomness affects exploration patterns. This algorithm is essential in many applications, including PageRank, recommendation systems, and Markov Chain Monte Carlo methods.

16.9. Exercise

16.9.1. Exercise 1

After implementing the Random Walk algorithm using NumPy, now it's time to understand the algorithm. What we want

[Saltar al contenido principal](#)

your own words, and try to execute each step of the algorithm, showing print statements for each step. This will help you understand the algorithm better and see how it works in practice.

16.9.2. Exercise 2

Apply the Random Walk algorithm to the synthetic graphs that you generated in the previous notebook, and discuss the results. How does the algorithm perform on these graphs? In what cases might Random Walks be more appropriate than BFS or DFS? Pay attention to the differences in path lengths and the nature of the paths found by each algorithm.

16.10. Submission

For the delivery of the practice 3, what you will be asked to do is to make a zip

[Saltar al contenido principal](#)

a first notebook containing the generation section, a second notebook containing BFS, a third notebook containing the DFS algorithm and a fourth notebook containing the Random Walk algorithm.

At the end the zip should be like this when you unzip it:

```
name_surname.zip
├── graph_exploration_bfs.ipynb
├── graph_exploration_dfs.ipynb
├── graph_exploration_rw.ipynb
└── graph_exploration_generation.ip
```